

实验二 组合电路设计实验报告

姓名：陈墨霏
班级：07112304

学号：1120233329
手机：13126146305

1. 实验题目

设计一个组合电路，输入一个 4 位的数字，输出一个 3 位的二进制数字，且输出数字的值近似等于输入数字值的平方根。例如，如果平方根的值等于 3.5 或者更大的值，则四舍五入记为 4。如果平方根的值小于 3.5 大于 2.5，则记为 3。

2. 实验约束

- 电路设计时只能使用与非门和或非门进行实现。
- 采用 Verilog 实现时使用结构化描述方式。

3. 电路设计

a) 规范化

该组合电路是一个实现对一个 4 位数字近似取平方根的功能的电路。

电路的输入是一个向量 $A(3:0)$ 。向量 A 有 4 位，分别为 $A(3)$ 、 $A(2)$ 、 $A(1)$ 、 $A(0)$ ，其中 $A(3)$ 为最高有效位。电路的输出是一个向量 $B(2:0)$ 。向量 B 有 3 位，分别是 $B(2)$ 、 $B(1)$ 、 $B(0)$ ，其中 $B(2)$ 为最高有效位。电路首先对输入向量 A 取平方根，之后对平方根进行近似，得到输出向量 B 。

例如，如果平方根的值等于 3.5 或者更大的值，则四舍五入记为 4。如果平方根的值小于 3.5 大于 2.5，则将其记为 3。

b) 形式化

根据规范化步骤中的描述，算数上可以使用以下步骤得到向量 B ：首先将向量 A 所对应的二进制数转化为十进制数，之后将向量 A 取平方根。对平方根进行十进制下到达个位的四舍五入近似，得到一个十进制整数。之后，将十进制整数转化为二进制数，得到相应的结果向量 B 。

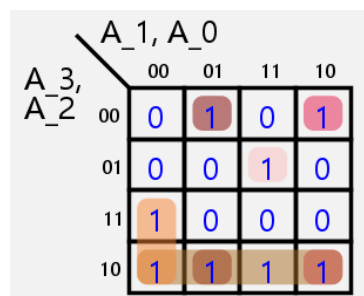
输入、输出可以用以下真值表来形式化描述和建模。

Table 1: 真值表

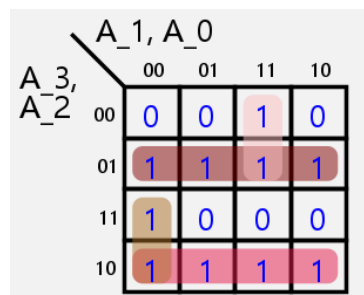
A(3)	A(2)	A(1)	A(0)	B(2)	B(1)	B(0)
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	1	0
0	1	0	0	0	1	0
0	1	0	1	0	1	0
0	1	1	0	0	1	0
0	1	1	1	0	1	1
1	0	0	0	0	1	1
1	0	0	1	0	1	1
1	0	1	0	0	1	1
1	0	1	1	0	1	1
1	1	0	0	0	1	1
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	1	0	0

c) 优化

根据真值表，绘制出相应的卡诺图。根据卡诺图，进行化简，得到相应的逻辑表达式。

Figure 1: $B(0)$

$$B(0) = \overline{A}BCD + A\overline{B} + A\overline{C}\overline{D} + \overline{B}\overline{C}D + \overline{B}C\overline{D}$$

Figure 2: $B(1)$

$$B(1) = \overline{A}B + \overline{A}CD + A\overline{B} + A\overline{C}\overline{D}$$

A ₃ , A ₂		A ₁ , A ₀			
		00	01	11	10
00	00	0	0	0	0
01	01	0	0	0	0
11	11	0	1	1	1
10	10	0	0	0	0

Figure 3: $B(2)$

$$B(2) = ABC + ABD$$

d) 工艺映射

该部分通过工艺映射, 将逻辑表达式转化为电路图, 并进一步转化为仅使用与非门和非门的电路图。

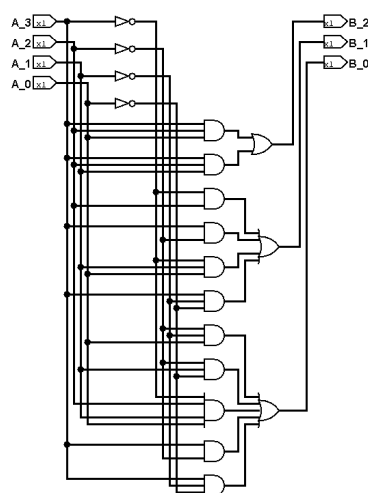


Figure 4: 根据表达式直接构建的电路图

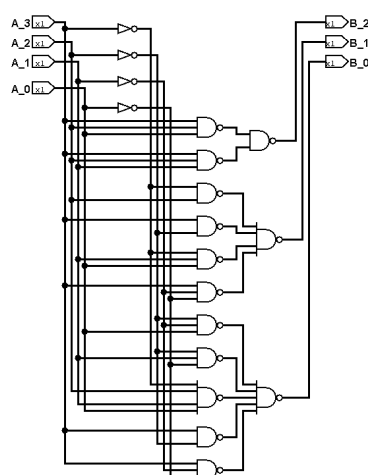


Figure 5: 仅使用与非门和非门的电路图

4. 电路实现

下面的数据流描述级 Verilog 代码实现了上述电路。

sqrt.v

```
`timescale 1ns / 1ps
module sqrt (
    input [3:0] A,
    output [2:0] B
);
    // 中间信号的导线
    wire s_logisimNet0, s_logisimNet1, s_logisimNet2, s_logisimNet3;
    wire s_logisimNet4, s_logisimNet5, s_logisimNet6, s_logisimNet7;
    wire s_logisimNet8, s_logisimNet9, s_logisimNet10, s_logisimNet11;
    wire s_logisimNet12, s_logisimNet13, s_logisimNet14, s_logisimNet15;
    wire s_logisimNet16, s_logisimNet17, s_logisimNet18, s_logisimNet19;
    wire s_logisimNet20, s_logisimNet21;

    // 输入连接
    assign s_logisimNet0 = A[2];
    assign s_logisimNet4 = A[1];
    assign s_logisimNet6 = A[3];
    assign s_logisimNet9 = A[0];

    // 输出连接
    assign B[0] = s_logisimNet15;
    assign B[1] = s_logisimNet8;
    assign B[2] = s_logisimNet20;

    // 逻辑门
    assign s_logisimNet1 = ~s_logisimNet0;
    assign s_logisimNet3 = ~s_logisimNet4;
    assign s_logisimNet2 = ~s_logisimNet9;
    assign s_logisimNet13 = ~s_logisimNet6;

    assign s_logisimNet5 = s_logisimNet6 & s_logisimNet0 & s_logisimNet9;
    assign s_logisimNet12 = s_logisimNet6 & s_logisimNet0 & s_logisimNet4;
    assign s_logisimNet16 = ~s_logisimNet6 & s_logisimNet0;
    assign s_logisimNet19 = s_logisimNet6 & ~s_logisimNet0;
    assign s_logisimNet11 = ~s_logisimNet6 & s_logisimNet4 & s_logisimNet9;
    assign s_logisimNet14 = s_logisimNet6 & ~s_logisimNet4 & ~s_logisimNet9;
    assign s_logisimNet17 = ~s_logisimNet0 & ~s_logisimNet4 & s_logisimNet9;
    assign s_logisimNet18 = ~s_logisimNet0 & s_logisimNet4 & ~s_logisimNet9;
    assign s_logisimNet21 = ~s_logisimNet6 & s_logisimNet0 & s_logisimNet4 &
s_logisimNet9;
    assign s_logisimNet7 = s_logisimNet6 & ~s_logisimNet0;
    assign s_logisimNet10 = s_logisimNet6 & ~s_logisimNet4 & ~s_logisimNet9;
    assign s_logisimNet20 = s_logisimNet5 | s_logisimNet12;
    assign s_logisimNet8 = s_logisimNet16 | s_logisimNet19 | s_logisimNet11 |
s_logisimNet14;
    assign s_logisimNet15 = s_logisimNet17 | s_logisimNet18 | s_logisimNet21 |
s_logisimNet7 | s_logisimNet10;

endmodule
```

5. 电路验证

a) TestBench

该部分通过编写 Verilog TestBench，对实现的电路模块进行功能验证。

testbench.v

```
`timescale 1ns / 1ps
module testbench();
    reg [3:0] A; // 4-bit 输入向量 A
    wire [2:0] B; // 3-bit 输出向量 B

    // 例化待测模块 sqrt
    sqrt uut (
        .A(A),
        .B(B)
    );

    // 定义数据输入
    initial begin
        // 测试用例 1
        A = 4'b0000; // 输入为 0
        #100;

        // 测试用例 2
        A = 4'b0001; // 输入为 1
        #100;

        // 测试用例 3
        A = 4'b0010; // 输入为 2
        #100;

        // 测试用例 4
        A = 4'b0100; // 输入为 4
        #100;

        // 测试用例 5
        A = 4'b1000; // 输入为 8
        #100;

        // 测试用例 6
        A = 4'b1111; // 输入为 15
        #100;

        $stop; // 停止仿真
    end
endmodule
```

b) 仿真结果

在 Vivado 中运行仿真，得到如下波形图。

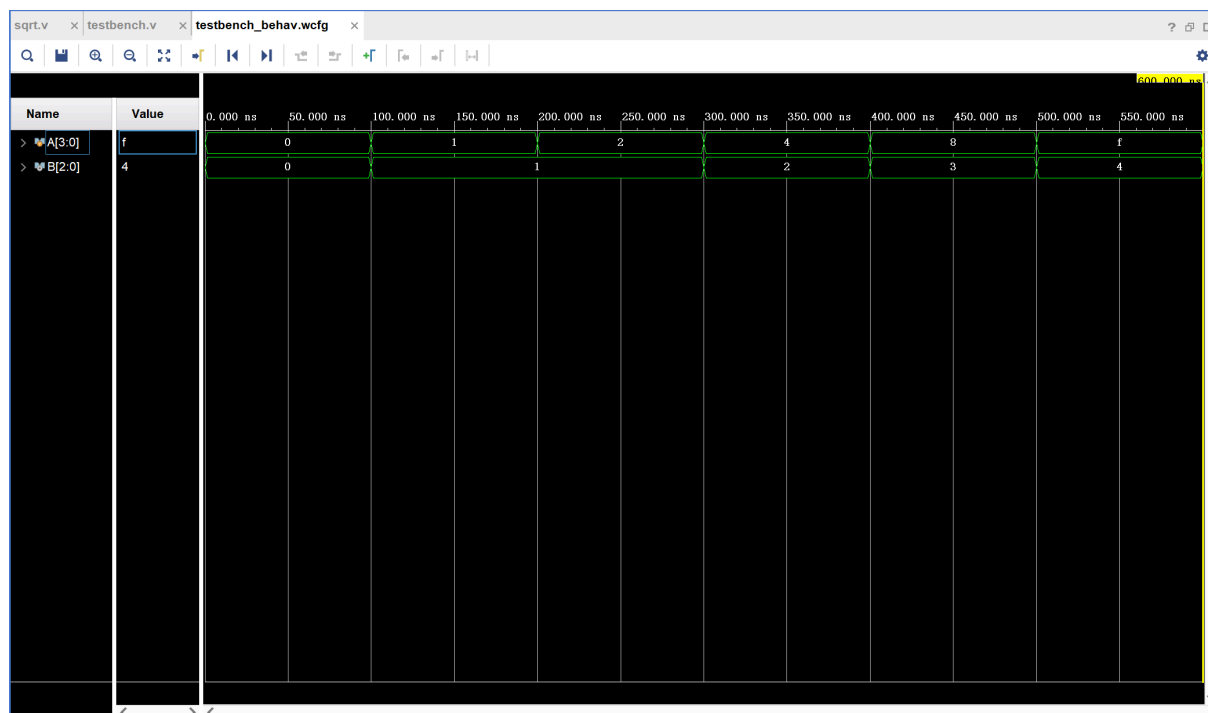


Figure 6: 仿真波形图

从波形图中可以看出，输入向量A的值与输出向量B的值之间存在着一定的关系。对于每一个输入向量A，输出向量B的值都近似等于输入向量A的平方根。这符合我们对于电路的功能预期。

6. 实验心得

在本次实验中，我学习了如何使用 Verilog 语言进行组合电路的设计和实现。通过对电路的规范化、形式化、优化和工艺映射等步骤，我掌握了组合电路设计的基本流程。同时，我也学会了如何使用 Vivado 进行电路的仿真和验证。通过本次实验，我对组合电路的设计和实现有了更深入的理解，也提高了自己的 Verilog 编程能力。

此外，我还学习并使用到了一个强大的数字电路设计工具：Logisim。Logisim 是一个开源的数字电路设计和仿真工具，它提供了一个直观的图形界面，可以方便地进行电路设计、电路图导出和逻辑仿真。通过使用 Logisim，我能够更直观地理解电路的工作原理，并且能够快速地进行电路的设计和验证。